

Day 6: SQL Notes Basic to Advanced

Today's Topic: SQL Clauses

- Group By
- Having
- Order By
- Where By

What are SQL Clauses?

- SQL clauses are the essential components of SQL (Structured Query Language) statements.
- They define the specific tasks you want the SQL engine to perform, such as retrieving data, filtering records, or modifying database tables.
- Each clause serves a unique purpose and is used in different contexts to manipulate or query the data stored in a relational database.
- Clauses define specific conditions within SQL queries.
- Common clauses include WHERE, GROUP BY, and ORDER BY.
- Used to refine query results based on criteria.

6.1 GROUP BY

- The *GROUP BY* clause in SQL is used to group rows that have the same values in specified columns into aggregated data.
- It is commonly used in conjunction with aggregate functions like *COUNT*, *SUM*, *AVG*, *MAX*, and *MIN* to perform calculations on each group of data.

How GROUP BY Works

- When you use the *GROUP BY* clause, SQL groups rows that have identical values in the specified columns and then performs any specified aggregate calculations on those groups.
- The result is a single row for each group, containing the grouped columns and the results of the aggregate functions.

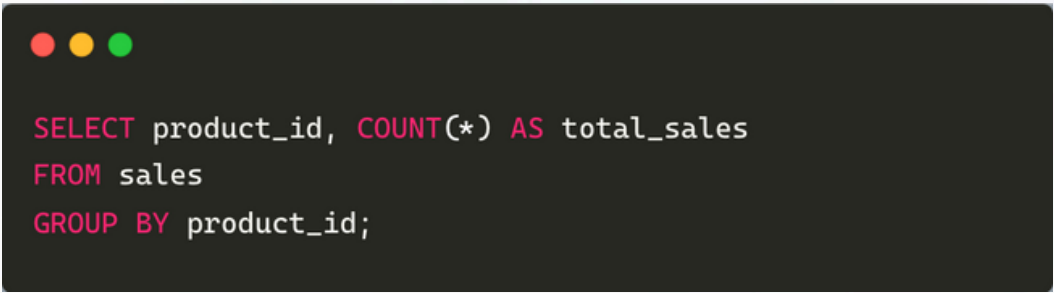
Syntax

```
SELECT column1, aggregate_function(column2)
FROM table_name
GROUP BY column1;
```

- `column1`: The column by which you want to group the data.
- `aggregate_function(column2)`: An aggregate function like `COUNT`, `SUM`, `AVG`, `MAX`, or `MIN` applied to another column.

Example 1: Counting Records in Each Group

Suppose you have a table called `sales` with columns `product_id`, `quantity`, and `sale_date`. You want to count how many times each product has been sold.

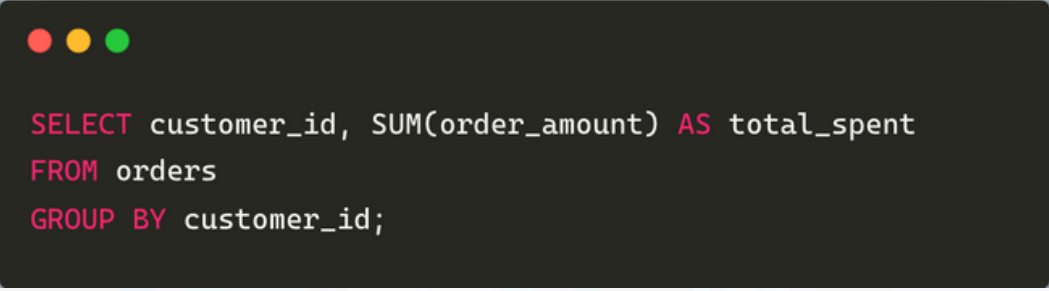
A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains a SQL query in a monospaced font.

```
SELECT product_id, COUNT(*) AS total_sales
FROM sales
GROUP BY product_id;
```

This query groups the rows by `product_id` and counts the number of records in each group.

Example 2: Summing Values in Each Group

Consider a table called `orders` with columns `customer_id`, `order_amount`, and `order_date`. To find out the total amount each customer has spent, you could use:



```
SELECT customer_id, SUM(order_amount) AS total_spent
FROM orders
GROUP BY customer_id;
```

This query groups the rows by `customer_id` and sums the `order_amount` for each customer.

6.2 HAVING

- The **HAVING** clause in SQL is used to filter groups of rows that are created by the **GROUP BY** clause.
- It is similar to the **WHERE** clause, but while **WHERE** is used to filter individual rows before any grouping is done, **HAVING** filters groups after the grouping has been performed.

How HAVING Works

- The **HAVING** clause is typically used in combination with aggregate functions (such as **COUNT**, **SUM**, **AVG**, **MAX**, **MIN**) to filter the results based on the aggregate values.
- This allows you to apply conditions to the summarized data.

Syntax:

```
SELECT column1, aggregate_function(column2)
FROM table_name
GROUP BY column1
HAVING condition;
```

- column1: The column by which you have grouped the data.
- aggregate_function(column2): An aggregate function applied to another column.
- condition: The condition you want to apply to the grouped data.

Example 1: Filtering by Aggregate Value

Suppose you have a table called sales with columns product_id, quantity, and sale_date. You want to find products that have been sold more than 10 times.

```
SELECT product_id, COUNT(*) AS total_sales
FROM sales
GROUP BY product_id
HAVING COUNT(*) > 10;
```

This query groups the rows by product_id, counts the number of sales for each product, and then filters out products with 10 or fewer sales.

6.3 ORDER BY

This query groups the rows by `product_id`, counts the number of sales for each product, and then filters out products with 10 or fewer sales.

How ORDER BY Works

- When you use the `ORDER BY` clause, SQL arranges the rows returned by a query according to the values in specified columns.
- You can sort data numerically, alphabetically, or by date, and you can specify multiple columns for sorting.

Syntax

```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column1 [ASC|DESC], column2 [ASC|DESC], ... ;
```

- `column1, column2, ...`: The columns by which you want to sort the data.
- `ASC`: Sorts the data in ascending order (this is the default).
- `DESC`: Sorts the data in descending order.

Example 1: Sorting by a Single Column

Suppose you have a table called `employees` with columns `last_name`, `first_name`, and `hire_date`. To get a list of employees sorted by `last_name` in ascending order:

```
SELECT last_name, first_name, hire_date
FROM employees
ORDER BY last_name ASC;
```

This query sorts the results by `last_name` alphabetically from A to Z.

Example 2: Sorting by Multiple Columns

If you want to sort the employees first by `last_name` in ascending order and then by `first_name` in descending order:

```
SELECT last_name, first_name, hire_date
FROM employees
ORDER BY last_name ASC, first_name DESC;
```

This query sorts the list by `last_name` and then by `first_name` within each `last_name`.

5.4 WHERE

- The WHERE clause in SQL is used to filter records based on specified conditions.
- It is one of the most fundamental clauses in SQL, allowing you to retrieve only the rows that meet certain criteria.
- The WHERE clause can be used in SELECT, UPDATE, DELETE, and other SQL statements to refine the results.

How WHERE Works

- The WHERE clause filters rows from a table based on the condition you specify. Only the rows that satisfy the condition are included in the result set.
- The condition can include operators like =, >, <, >=, <=, <>, BETWEEN, LIKE, IN, IS NULL, AND, OR, and NOT.

Syntax

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

- column1, column2, ...: The columns you want to retrieve.
- table_name: The table from which you want to retrieve the data.
- condition: The criteria that the rows must meet to be included in the result set.

Example 1: Filtering by a Single Condition

Suppose you have a table called employees with columns employee_id, last_name, department, and salary. You want to retrieve all employees who work in the "Sales" department.

```
SELECT employee_id, last_name, department, salary
FROM employees
WHERE department = 'Sales';
```

This query returns only the rows where the department column is equal to "Sales".

Example 2: Using Multiple Conditions with AND

If you want to find all employees in the "Sales" department who have a salary greater than 50,000:

```
SELECT employee_id, last_name, department, salary
FROM employees
WHERE department = 'Sales' AND salary > 50000;
```

Summary

- **GROUP BY Clause:** Groups rows by specified columns for aggregation. Used with aggregate functions.
- **HAVING Clause:** Filters results of **GROUP BY** based on aggregate conditions. Applied after grouping.
- **ORDER BY Clause:** Sorts query results by one or more columns in ascending or descending order.
- **WHERE Clause:** Filters individual rows based on specified conditions before any grouping or sorting.

Day 7: SQL Joins

Will Post Tomorrow Follow For More

Want This PDF?

Join Our Telegram and
Download **100+ PDFs** and
Handwritten Notes

